UNIVERSITÄT BERN

3D Geometry Processing
Exercise 04: Curves

Submission deadline: 24.03.2026, 13:00

Late submissions are not accepted

Submission rules

You should submit a single `.zip` compressed file named `Ex`n`-GroupMemberNames.zip` where n is the number of the current exercise sheet. *Please order the names alphabetically by last name to make life easier for us :)* The file should contain:

- In programming exercises: all code files you changed, and **only** the files you changed (headers and source).
- A single file (`.txt`, `.pdf` or `.md`) with the following contents:
  - Names of all team members,
  - Any difficulties encountered and how you resolved them,
  - Optionally any comments about the exercise.
  - Your solutions to the theoretical exercises (if any)
  - (Only in case you used any AI assistants:) List for which purposes/problems you used them, and your judgement of where this was helpful, useless, or misleading.
- (Only in case you used any AI assistants:) Include all your chat transcripts as separate files in an archive `AI-transcripts.zip` inside the main `.zip`.

1 Theory Exercise (2 pts)

1.1 Curvature of Curves (1 pt)

Match each curve from Figure 1 with the corresponding curvature function:

$$k_1(s) = \frac{s^2 - 1}{s^2 + 1}, \quad k_2(s) = s, \quad k_3(s) = s^3 - 4s, \quad k_4(s) = \sin(s)s. \tag{1}$$

Assume that each of the curves (and corresponding curvature functions) have the whole real line in their domain. That is $\gamma_i : \mathbb{R} \to \mathbb{R}^2, k_i : \mathbb{R} \to \mathbb{R}$ If, for example, the curvature function $k_1(s)$ corresponds to the curve **a**, write **1-a** as your solution. Briefly explain your answers.

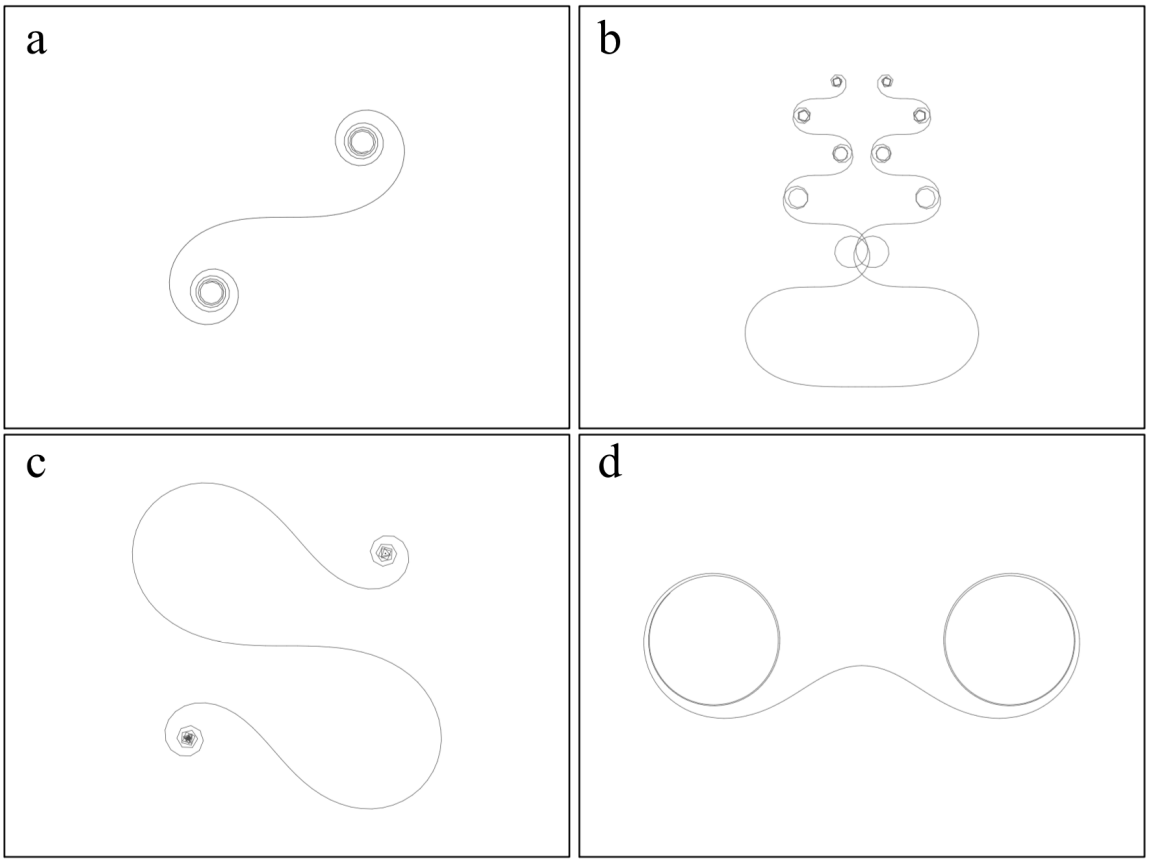

Figure 1: *Curves reconstructed from given curvature functions $k_i(s)$.*

1.2 Surface Area (1 pt)

King Archimedes wants to renovate his palace. The most striking structure is a spherical half-dome of 20m in diameter that covers the great hall. The king wants to cover this dome in a layer of pure gold. He has decided to split the work into two parts, each one covering a vertical slice of the dome of the same height (see Figure 2). For each part he hires different people and gives them 700kg of gold. The task is to cover the surface of one vertical slice with a layer of gold of 0.1mm thickness (the thickness is sufficiently small such that surface area times thickness is a good approximation of the actual volume). The amount of gold that is left over is the salary for doing the job. Which slice should you pick if you want to make the most profit? Explain your answer in `TheoryExercise.pdf`. How does your answer change when you have n slices instead of just two?

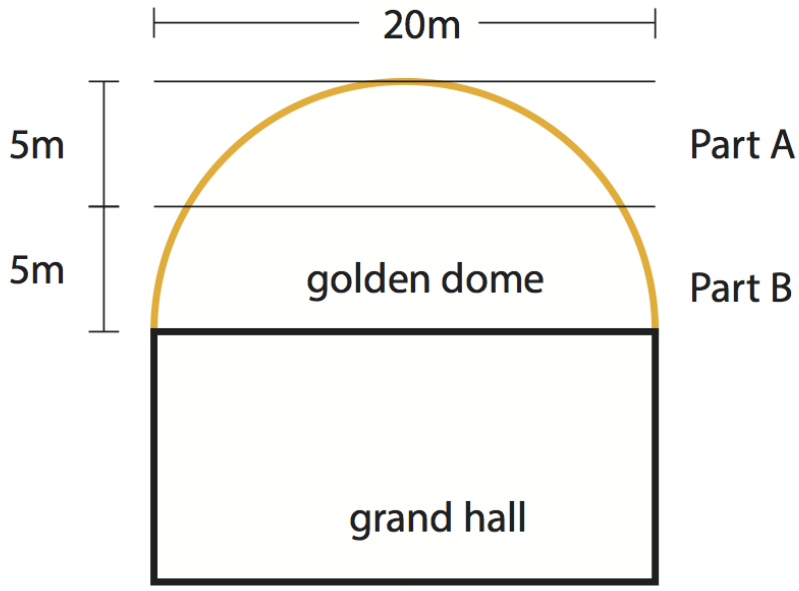

Figure 2: *Sketch of King Archimedes dome.*

2 Coding Exercise: Curve smoothing (2 pts)

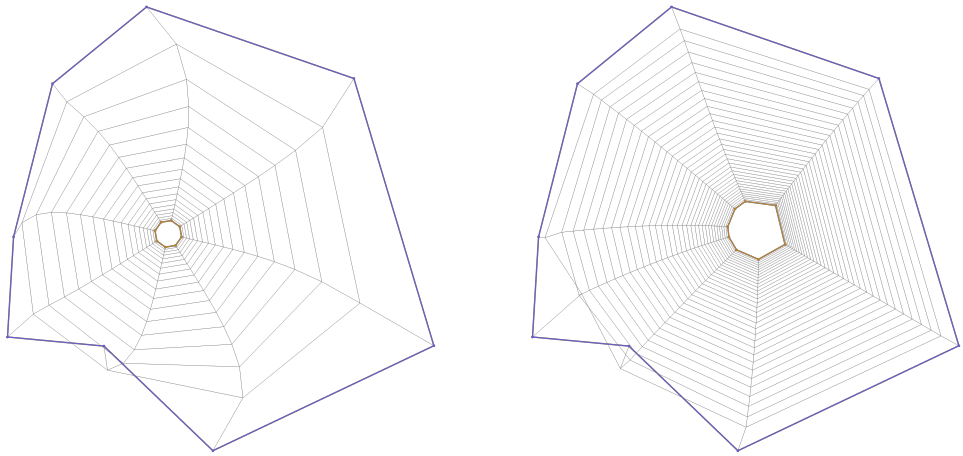

Figure 3: *Curve shortening flow discretized using uniform Laplacian (left) and osculating circle (right)*

Overview

In this task, we consider curves in $\mathbb{R}^3$ represented discretely as polylines with vertices $\{V_i\}_{i=0}^{M-1}$. The curves can be open (with boundaries at V_0 and V_{M-1}) or closed (with a line segment between V_0 and V_{M-1}).

We will implement two discretizations of the curve-shortening flow which can be used for curve smoothing.

The curve-shortening flow is a partial differential equation that describes the evolution of a curve that moves points along the normal (oriented "inwards" towards the center of the osculating circle) with speed proportional to the curvature:

$$\frac{\partial C}{\partial t} = \frac{\partial^2 C}{\partial s^2} = \kappa n, \tag{2}$$

where $C : D \to \mathbb{R}^3$ is an arc-length-parameterized curve.

Don't worry too much about this PDE, as this process can also be practically understood purely discretely in terms of polyline vertices.

We will compute multiple discrete time steps of the curve evolution, initializing with vertex positions $V_i^0 = V_i$, computing V_i^{k+1} based on the values V_i^k. For curves with boundaries, V_0 and V_{M-1} positions are fixed.

Download `Plugin-Ex04-CurveSmoothing.zip` from ILIAS and extract it into your `Package-GP` folder. For this exercise you will need to fill in the missing code in the file `CurveSmoothingPlugin.cc`. Read the supplied code carefully and fill in the missing parts marked as TODOs.

Inputs

To test the smoothing algorithms on different examples, you can generate random curves from within the plugin, or import outputs from Exercise 03 (Iso-Contouring). For that, you will have to mark the iso-contour output object as *Target* in the DataControl plugin, then press the *Import* button in the CurveSmoothing plugin.

2.1 Uniform Laplacian Smoothing (0.75 points)

Move every vertex towards the centroid of the two neighbors. More specifically, every non-boundary vertex position V_i is updated to move towards the average position of its neighbors according to

$$V_i^{k+1} = (1-\varepsilon) \cdot V_i^k + \varepsilon \cdot \frac{1}{2}\left(V_{i-1}^k + V_{i+1}^k\right), \tag{3}$$

where ε is a small time step, and indexing is understood modulo M for closed curves.

Implement this smoothing by filling in the `smoothLaplacian()` function and experiment with different open and closed curves and time steps.

2.2 Length preservation (0.25 points)

In your experiments, you will – in the absence of bugs, and with a sufficiently small ε – likely notice that the curve-shortening flow indeed shortens the curves. The continuous flow generated using Eq. (2) can be shown to always converge to a "round" point for closed curves, and our discretized version exhibits similar behavior. For many practical purposes however (including investigating what shapes it converges to), this shrinkage is not desired, so we want to counteract this effect by rescaling the curves.

Implement `rescaleCurve()` which takes a curve and adjusts its vertices by uniform scaling such that the total length equals a given target length. This target length is computed in `smoothCurve()` using `polylineLength()`, the latter of which you'll also have to implement.

To avoid drift, ensure that the center of the curve (computed using `polylineCenter()`) stays constant.

2.3 Osculating circle smoothing (1 point)

An alternative approach with a slightly more involved implementation considers a discretization of the osculating circle as the circumcircle of three consecutive vertices[1].

Consider a discrete osculating circle at vertex V_i as a circumscribed circle O_i of a triangle defined with vertices V_{i-1}, V_i and V_{i+1}. The direction of $C_i - V_i = R_i \cdot n$ is now a normal vector of the curve at V_i, with magnitude $R_i = 1/\kappa_i$. We want to shift every non-boundary vertex position V_i

[1] this is related to the notion of Menger curvature: https://en.wikipedia.org/wiki/Menger_curvature

towards the center C_i of O_i by a distance proportional to κ. Therefore we divide $C_i - V_i$ by R^2 to obtain κn:

$$V_i^{k+1} = V_i^k + \varepsilon \cdot \delta \cdot \kappa n \tag{4}$$

$$= V_i^k + \varepsilon \cdot \delta \cdot \frac{C_i - V_i}{\|C_i - V_i^2\|} \tag{5}$$

Here ε is a small time step, and $\delta = \ell_{\min}^2$ is an additional scaling factor ensuring scale invariance, with $\ell_{\min}$ the length of the shortest edge segment.

Implement this algorithm by completing the function `smoothCurvatureFlow()`. Experiment with different open and closed curves and time steps. Compare the behavior with uniform Laplacian smoothing.